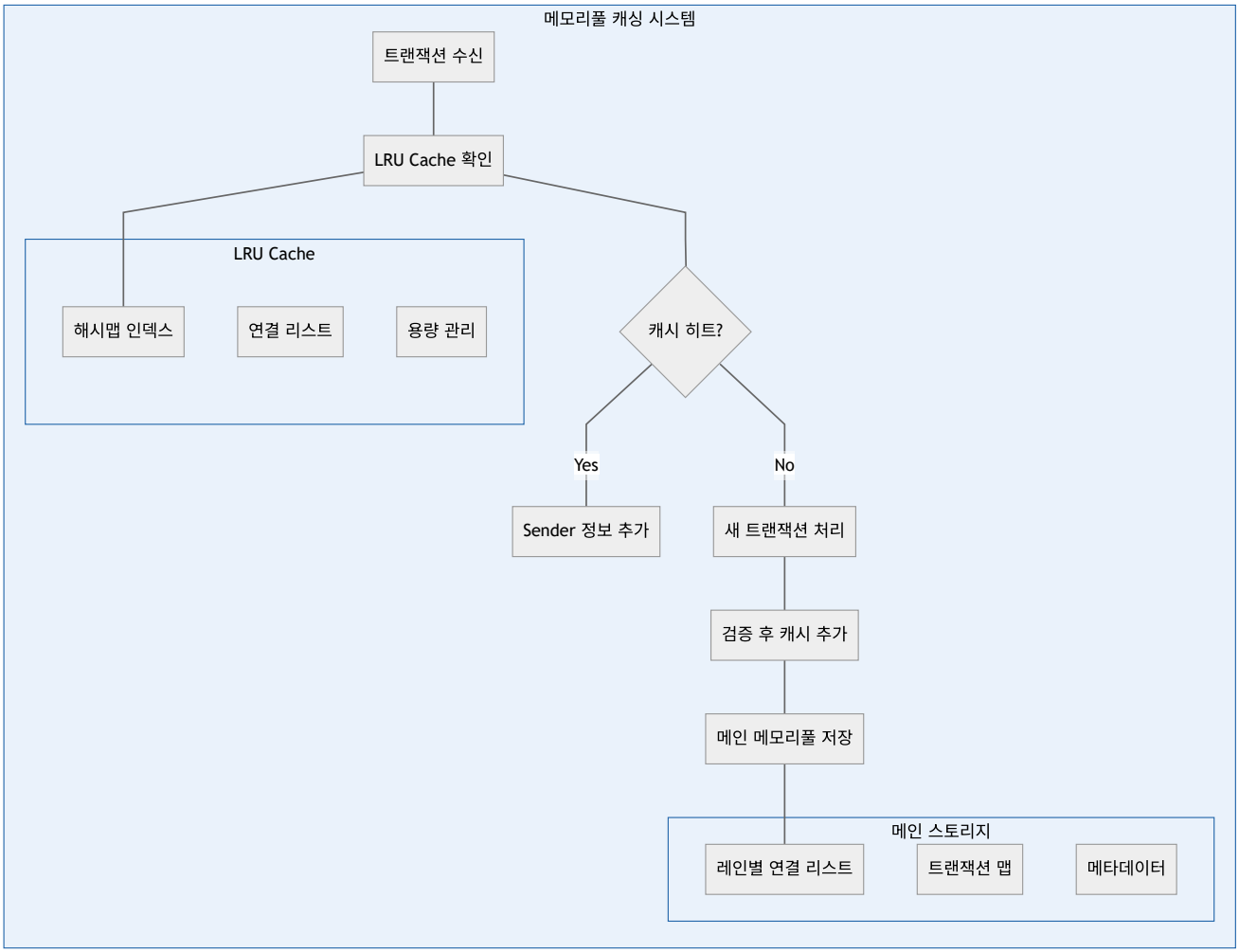
메모리풀 캐싱 및 관리 전략

개요

CometBFT 메모리풀은 효율적인 트랜잭션 관리를 위해 다층적인 캐싱 전략과 메모리 관리 기법을 사용합니다. 이 문서는 메모리풀의 캐싱 메커니즘, 크기 관리, 가비지 컬렉션, 그리고 메모리 사용량 최적화 전략을 상세히 설명합니다.

캐싱 아키텍처

전체 캐싱 구조



LRU 캐시 구현

캐시 구조

CometBFT는 LRU(Least Recently Used) 알고리즘을 사용하여 중복 트랜잭션을 효율적으로 필터링합니다:

```
type LRUTxCache struct {
mtx cmtsync.Mutex // 동시성 보호
size int // 최대 캐시 크기
cacheMap map[types.TxKey]*list.Element // 빠른 검색을 위한 해시맵
list *list.List // LRU 순서 관리용 연결 리스트
}
```

캐시 동작 원리

1. 트랜잭션 추가 (Push)

```
func (c *LRUTxCache) Push(tx types.Tx) bool {
c.mtx.Lock()
defer c.mtx.Unlock()
key := tx.Key()
// 이미 존재하는 트랜잭션인지 확인
if moved, ok := c.cacheMap[key]; ok {
// 최근 사용으로 표시 (리스트 끝으로 이동)
c.list.MoveToBack(moved)
return false // 이미 존재함
}
// 캐시 용량 초과 시 가장 오래된 항목 제거
```

```go
if c.list.Len() >= c.size {
front := c.list.Front()
if front != nil {
frontKey := front.Value.(types.TxKey)
delete(c.cacheMap, frontKey)
c.list.Remove(front)
}
}
// 새 트랜잭션 추가
e := c.list.PushBack(key)
c.cacheMap[key] = e
return true // 새로 추가됨
}
```

2. 캐시 조회 (Has)

```go
func (c *LRUTxCache) Has(tx types.Tx) bool {
c.mtx.Lock()
defer c.mtx.Unlock()
_, ok := c.cacheMap[tx.Key()]
return ok
}
```

3. 트랜잭션 제거 (Remove)

```
func (c *LRUTxCache) Remove(tx types.Tx) {
c.mtx.Lock()
defer  c.mtx.Unlock()
key := tx.Key()
if  e := c.cacheMap[key];  e != nil {
delete(c.cacheMap, key)
c.list.Remove(e)
}
}
```

캐시 정책

무효한 트랜잭션 캐시 정책

```
// 설정에 따른 캐시 제거 정책
func (mem *CListMempool) tryRemoveFromCache(tx types.Tx) {
if !mem.config.KeepInvalidTxsInCache {
mem.forceRemoveFromCache(tx)
}
}

func (mem *CListMempool) forceRemoveFromCache(tx types.Tx) {
```

```
mem.cache.Remove(tx)

}
```

- **KeepInvalidTxsInCache = false**: 무효한 트랜잭션을 즉시 캐시에서 제거
- **KeepInvalidTxsInCache = true**: 무효한 트랜잭션도 캐시에 유지 (나중에 유효해질 가능성 고려)

메모리풀 크기 관리

전체 용량 제한

메모리풀은 다음과 같은 제한을 통해 메모리 사용량을 관리합니다:

```
type MempoolConfig struct {

Size  int  `mapstructure:"size"`  // 최대 트랜잭션 개수 (기본: 5,000)

MaxTxsBytes  int64  `mapstructure:"max_txs_bytes"`  // 전체 최대 크기 (기본: 64MB)

MaxTxBytes  int  `mapstructure:"max_tx_bytes"`  // 단일 트랜잭션 최대 크기 (기본: 1

CacheSize  int  `mapstructure:"cache_size"`  // 캐시 크기 (기본: 10,000)

}
```

용량 확인 로직

```
func (mem *CListMempool) isFull(txSize  int) error {

memSize := mem.Size()

txsBytes := mem.SizeBytes()

// 트랜잭션 개수 또는 전체 크기 제한 확인
```

```go
if memSize >= mem.config.Size ||
uint64(txSize)+uint64(txsBytes) > uint64(mem.config.MaxTxsBytes) {
return ErrMempoolIsFull{
NumTxs: memSize,
MaxTxs: mem.config.Size,
TxsBytes: txsBytes,
MaxTxsBytes: mem.config.MaxTxsBytes,
}
}
// 재검증 중인 경우 추가 제한
if mem.recheck.consideredFull() {
return ErrRecheckFull
}
return nil
}
```

레인별 용량 분할

각 레인은 전체 메모리풀 용량을 균등하게 분할하여 사용합니다:

```go
func (mem *CListMempool) isLaneFull(txSize int, lane LaneID) error {
laneTxs, laneBytes := mem.LaneSizes(lane)
// 레인별 용량 계산 (전체 용량을 레인 수로 균등 분할)
laneTxsCapacity := mem.config.Size / len(mem.sortedLanes)
```

```
laneBytesCapacity := mem.config.MaxTxsBytes / int64(len(mem.sortedLanes))

if  laneTxs > laneTxsCapacity ||

int64(txSize)+laneBytes > laneBytesCapacity {

return ErrLaneIsFull{

Lane: lane,

NumTxs: laneTxs,

MaxTxs: laneTxsCapacity,

Bytes: laneBytes,

MaxBytes: laneBytesCapacity,

}

}

return  nil

}
```

가비지 컬렉션 전략

자동 정리 메커니즘

1. 블록 커밋 시 정리

```
func (mem *CListMempool) Update(height  int64, txs types.Txs, txResults []*abci.

// 커밋된 트랜잭션 처리

for  i, tx := range  txs {

if  txResults[i].Code == abci.CodeTypeOK {
```

```
// 성공한 트랜잭션은 캐시에 유지 (중복 방지)
mem.addToCache(tx)
} else {
// 실패한 트랜잭션은 캐시에서 제거
mem.tryRemoveFromCache(tx)
}
// 메모리풀에서 제거
if err := mem.RemoveTxByKey(tx.Key()); err != nil {
mem.logger.Debug("Committed transaction not in local mempool")
}
}
return nil
}
```

2. 재검증 중 무효화된 트랜잭션 정리

```
func (mem *CListMempool) handleRecheckTxResponse(tx types.Tx) func(res *abci.Res
return func(r *abci.Response) error {
res := r.GetCheckTx()
// 무효해진 트랜잭션 제거
if res.Code != abci.CodeTypeOK || postCheckErr != nil {
if err := mem.RemoveTxByKey(tx.Key()); err != nil {
return err
```

```
}
// 캐시에서도 제거
mem.tryRemoveFromCache(tx)
// 메트릭 업데이트
mem.metrics.EvictedTxs.Add(1)
}
return nil
}
}
```

메모리 해제 최적화

연결 리스트 요소 분리

```
func (mem *CListMempool) RemoveTxByKey(txKey types.TxKey) error {
mem.txsMtx.Lock()
defer mem.txsMtx.Unlock()
elem, ok := mem.txsMap[txKey]
if !ok {
return ErrTxNotFound
}
memTx := elem.Value.(*mempoolTx)
// 연결 리스트에서 제거
```

```
mem.lanes[memTx.lane].Remove(elem)

elem.DetachPrev() // 이전 요소와의 연결 해제로 GC 최적화

// 인덱스 및 통계 업데이트

delete(mem.txsMap, txKey)

mem.txsBytes -= int64(len(memTx.tx))

mem.numTxs--

mem.laneBytes[memTx.lane] -= int64(len(memTx.tx))

return  nil

}
```

메모리 사용량 최적화

1. 효율적인 데이터 구조

트랜잭션 키 최적화

```
// SHA256 해시를 고정 크기 배열로 사용

type TxKey [sha256.Size]byte


// 트랜잭션에서 키 생성

func (tx Tx) Key() TxKey {

return  sha256.Sum256(tx)

}
```

메타데이터 최소화

```go
type mempoolTx struct {
height int64 // 8 bytes
gasWanted int64 // 8 bytes
tx types.Tx // 가변 크기 (실제 트랜잭션 데이터)
lane LaneID // string (작은 크기)
seq int64 // 8 bytes
timestamp time.Time // 24 bytes
senders sync.Map // 동적 크기 (피어 ID 맵)
}
```

2. 메모리 풀링

객체 재사용

```go
// 메시지 객체 풀링 (실제 구현에서는 sync.Pool 사용 가능)
var messagePool = sync.Pool{
New: func() interface{} {
return &protomem.Message{}
},
}

func getMessage() *protomem.Message {
return messagePool.Get().(*protomem.Message)
}
```

```go
func putMessage(msg *protomem.Message) {
msg.Reset()
messagePool.Put(msg)
}
```

3. 지연 로딩 및 압축

트랜잭션 데이터 압축 (선택적)

```go
// 큰 트랜잭션의 경우 압축 저장 고려
type compressedTx struct {
originalSize int
compressed []byte
}

func (ctx *compressedTx) Decompress() (types.Tx, error) {
// 압축 해제 로직
return decompress(ctx.compressed)
}
```

성능 모니터링

메트릭 수집

```go
type Metrics struct {
// 크기 관련 메트릭
Size metrics.Gauge // 현재 트랜잭션 개수
SizeBytes metrics.Gauge // 현재 메모리 사용량
LaneSize metrics.Gauge // 레인별 트랜잭션 개수
LaneBytes metrics.Gauge // 레인별 메모리 사용량
// 캐시 관련 메트릭
AlreadyReceivedTxs metrics.Counter // 캐시 히트 수
EvictedTxs metrics.Counter // 제거된 트랜잭션 수
// 성능 관련 메트릭
TxSizeBytes metrics.Histogram // 트랜잭션 크기 분포
TxLifeSpan metrics.Histogram // 트랜잭션 생존 시간
}
```

메트릭 업데이트

```go
func (mem *CListMempool) updateSizeMetrics(laneID LaneID) {
laneTxs, laneBytes := mem.LaneSizes(laneID)
label := string(laneID)
// 레인별 메트릭
mem.metrics.LaneSize.With("lane", label).Set(float64(laneTxs))
mem.metrics.LaneBytes.With("lane", label).Set(float64(laneBytes))
// 전체 메트릭
```

```
mem.metrics.Size.Set(float64(mem.Size()))

mem.metrics.SizeBytes.Set(float64(mem.SizeBytes()))

}
```

설정 최적화 가이드

캐시 크기 조정

```
[mempool]

# 캐시 크기 설정 (트랜잭션 개수 기준)

cache_size = 10000


# 무효한 트랜잭션 캐시 유지 여부

keep-invalid-txs-in-cache = false
```

권장사항:

- **높은 트래픽 환경**: `cache_size = 50000` 이상
- **메모리 제한 환경**: `cache_size = 5000` 이하
- **동적 환경**: `keep-invalid-txs-in-cache = false`

메모리풀 크기 조정

```
[mempool]

# 최대 트랜잭션 개수
```

```
size = 5000


# 전체 메모리풀 최대 크기 (바이트)
max_txs_bytes = 67108864 # 64MB


# 단일 트랜잭션 최대 크기 (바이트)
max_tx_bytes = 1048576 # 1MB
```

권장사항:

- **고성능 노드**: `size = 10000`, `max_txs_bytes = 128MB`
- **일반 노드**: `size = 5000`, `max_txs_bytes = 64MB`
- **경량 노드**: `size = 1000`, `max_txs_bytes = 16MB`

문제 해결

일반적인 메모리 이슈

1. 메모리 누수 감지

```
# 메모리 사용량 모니터링
curl -s http://localhost:26657/status | jq '.result.sync_info'


# 메모리풀 상태 확인
curl -s http://localhost:26657/num_unconfirmed_txs
```

2. 캐시 효율성 확인

```
// 캐시 히트율 계산
hitRate := float64(alreadyReceivedTxs) / float64(totalTxRequests)
if hitRate < 0.3 {
// 캐시 크기 증가 고려
log.Warn("Low cache hit rate", "rate", hitRate)
}
```

3. 메모리풀 정리

```
# 메모리풀 플러시 (개발/테스트 환경에서만)
curl -X POST http://localhost:26657/unsafe_flush_mempool
```

모범 사례

1. 용량 계획

- **트랜잭션 크기 분석**: 평균 트랜잭션 크기를 기반으로 용량 설정
- **피크 트래픽 고려**: 최대 예상 트래픽의 2-3배 여유 용량 확보
- **메모리 제한 준수**: 시스템 전체 메모리의 10-20% 이내로 제한

2. 모니터링 설정

- **알림 임계값**: 메모리풀 사용률 80% 이상 시 알림
- **성능 추적**: 트랜잭션 처리 지연시간 모니터링

- **캐시 효율성**: 캐시 히트율 주기적 확인

3. 정기 유지보수

- **로그 분석**: 메모리풀 관련 에러 로그 정기 검토
- **설정 최적화**: 트래픽 패턴 변화에 따른 설정 조정
- **성능 테스트**: 부하 테스트를 통한 최적 설정 검증

참고 자료

- 메모리풀 개요
- 트랜잭션 처리 과정
- 성능 튜닝 가이드